Bloom Filters

Theory, Variants, and Modern Networked Applications

Elad Mani

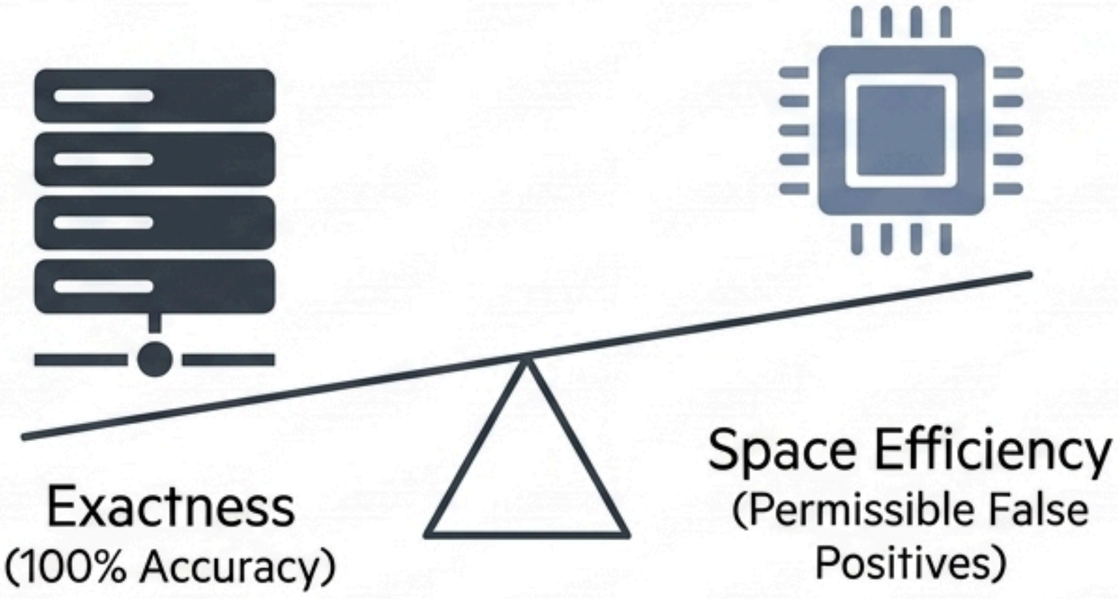
Introduction to Bloom Filters

- Space-efficient, probabilistic data structure.
- Tests for approximate set membership.
- Guarantees NO false negatives.
- Allows for controllable false-positive probability.
- Does not explicitly store items.

The Guiding Principle of Probabilistic Sets

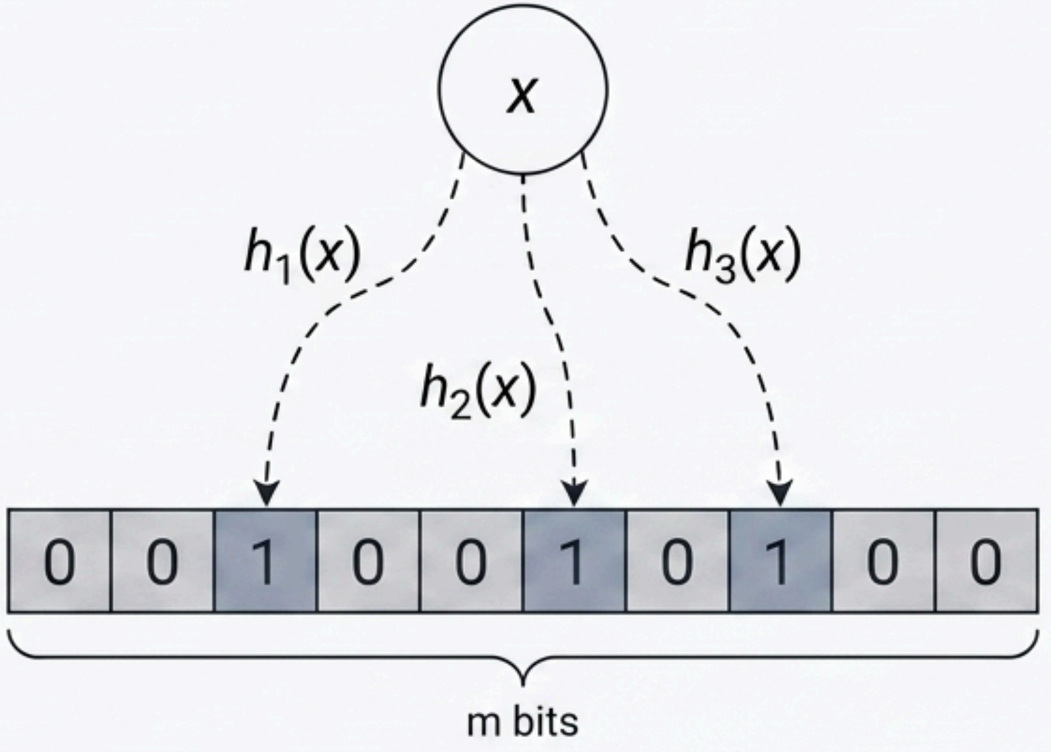
> “Wherever a list or set is used, and space is at a premium, consider using a Bloom filter if the effect of false positives can be mitigated.” — Broder & Mitzenmacher

Hash tables require O(1) slots scaling with key size. Prohibitive for massive data universes.



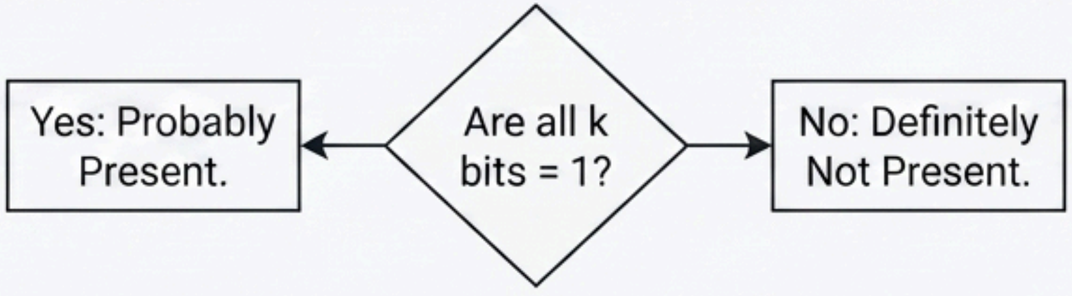
Zero false negatives. Controllable false positive probability (f). Negligible overhead per item.

Standard Bloom Filters: The Mechanism



Notation Key

n: Number of elements in the set.
m: Length of the bit array.
k: Number of independent hash functions.



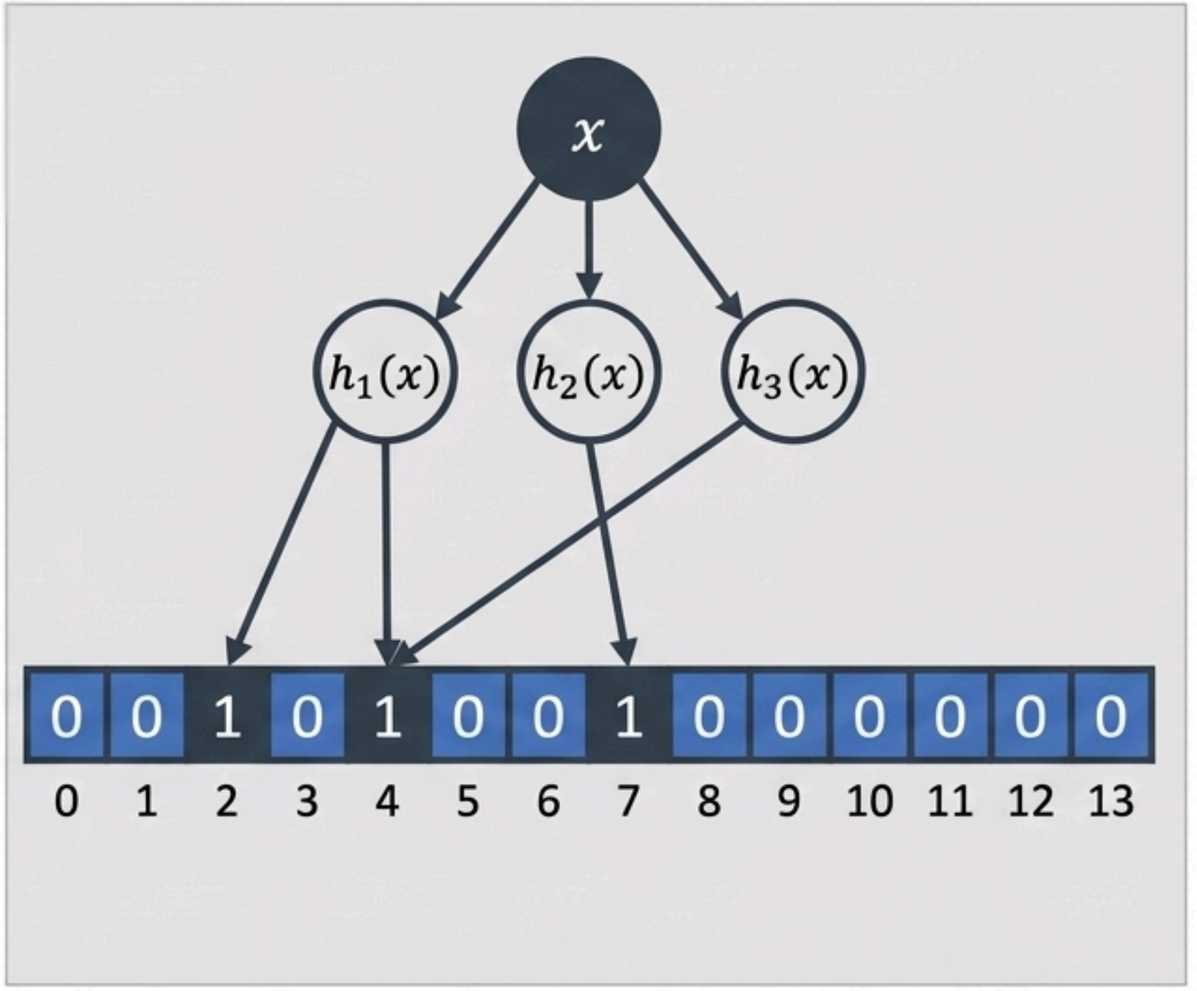
Architecture of a Standard Bloom Filter

The Universe: n elements represented within a bit array of length m.

The Engine: k independent hash functions mapping elements to bounded indices.

Insert (x): Set bits at indices $h_i(x)$ to 1 for $i \in \{1, \ldots, k\}$.

Query (y): If any bit at $h_i(y)$ is 0
→ Definitely Not Present.
If all are 1 → Probably Present.



Bloom Filters vs. Hash Tables

	Structural Overhead	Storage Type	Space Complexity	Error Guarantees
Hash Tables	Requires O(1) slots, but massive bit-width per slot.	Explicit storage of original keys or pointers.	Scales with the size of the address space/item universe.	100% deterministic accuracy.
Bloom Filters	Constant bits per element.	No key storage (items are not explicitly stored).	$\approx 1.44 \log_2(1/\varepsilon)$ bits per item.	1-sided error (false positives allowed).

Bloom filters scale gracefully for massive networked sets where explicitly storing keys is physically prohibitive.

Calculating The False Positive Probability

Assumptions: Assuming perfect randomness, the probability that a specific bit remains 0 after a single hash operation is (1-1/m). After inserting n elements, each hashed k times, the exact probability q that a specific bit remains 0 is:

Step 1: Discrete Exact Probability (q)

$$q = \left(1 - \frac{1}{m}\right)^{kn} \quad (1)$$

The exact probability that a specific bit remains 0 after n elements are hashed k times.

Step 2: Continuous Approximation (p)

$$p = e^{-kn/m} \quad (2)$$

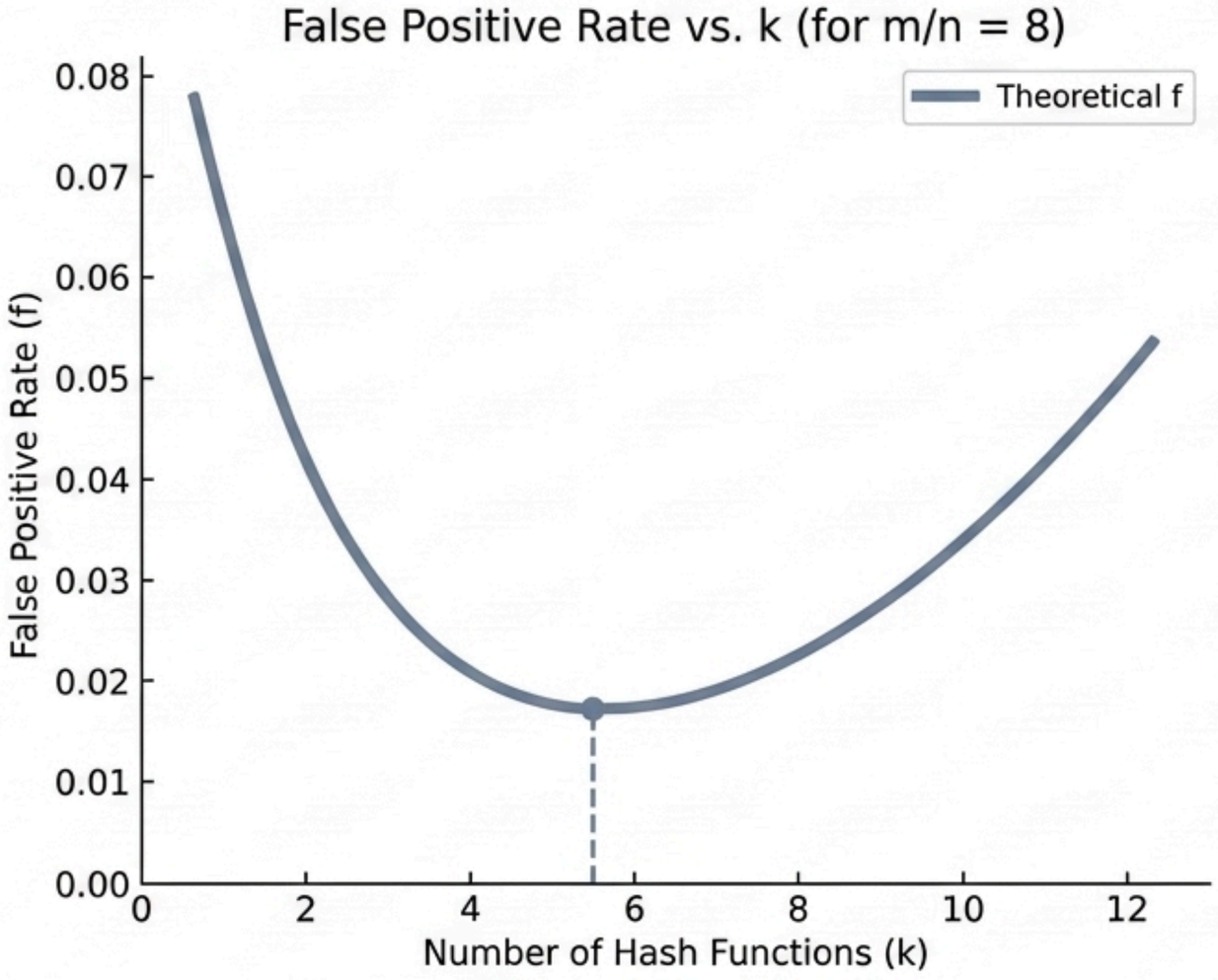
As m approaches infinity, the standard limit applies. This elegant closed-form is used to mathematically optimize the filter.

Step 3: False Positive Rate (f)

$$f = (1 - \alpha)^k \approx (1 - p)^k = \left(1 - e^{-kn/m}\right)^k$$

The final probability that all k bit positions for a non-member happen to already be set to 1 by coincidence.

Minimizing Error: The Optimal Hash Count



Derivation Insight: We find the optimal k by minimizing the False Positive Rate, $f = (1 - e^{-kn/m})^k$. This occurs when the probability of a bit being zero, $p = e^{-kn/m}$, is exactly 1/2. Solving $e^{-km} = 1/2$ for k yields the formula:

$$k_{opt} = \frac{m}{n} * \ln(2)$$

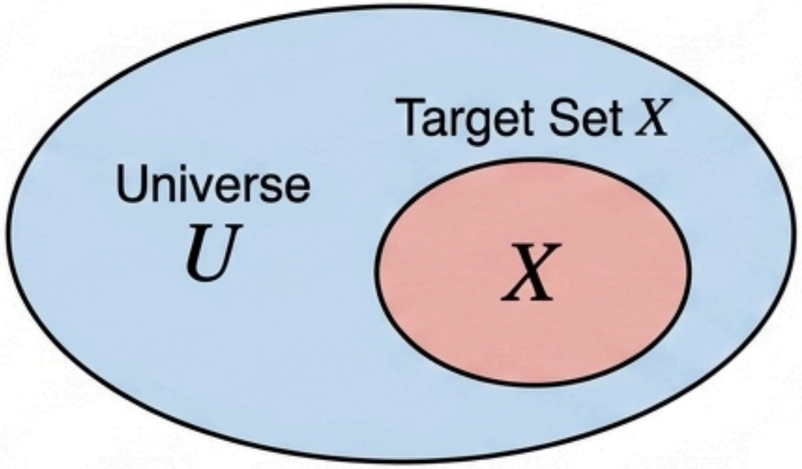
Round Down	Round Up
For **Speed:** Choosing a smaller k reduces the number of hash calculati-tons and memory probes required per query.	For **Safety:** Choosing a larger k ensures strict adherence to a maximum allowable false-positive boundary.

Insight: The curve is remarkably flat at its minimum, meaning integer rounding imposes a negligible performance penalty.

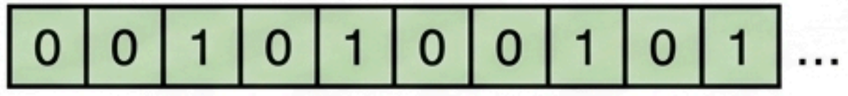
The Information-Theoretic Limit

The Information in its plant are simpl̈r to networks to setthat activite the first manctite objects are expossible and coves the bitarray as ε bits, and perfected $n+$ approximation sets.

Constraint 1: Total Possible Sets

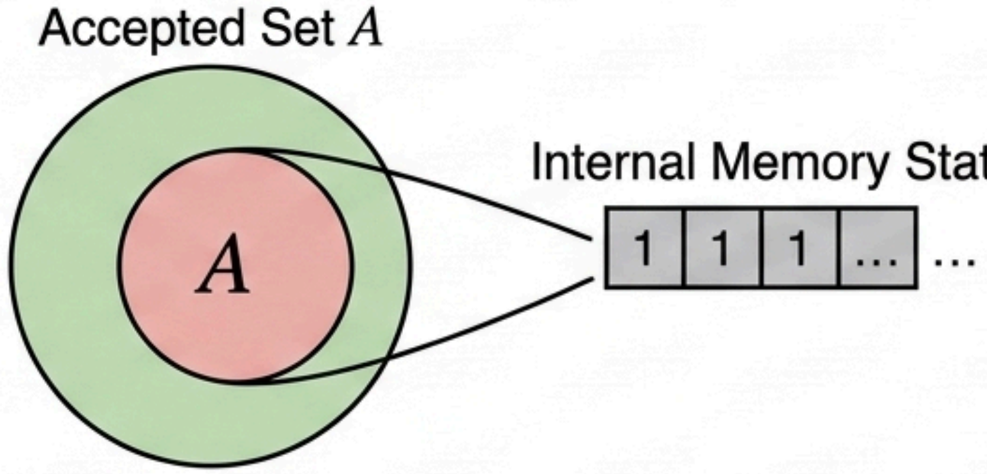


There are exactly $\binom{u}{n}$ possible sets.



m bits $= 2^m$ internal states

Constraint 2: The Power of Approximation



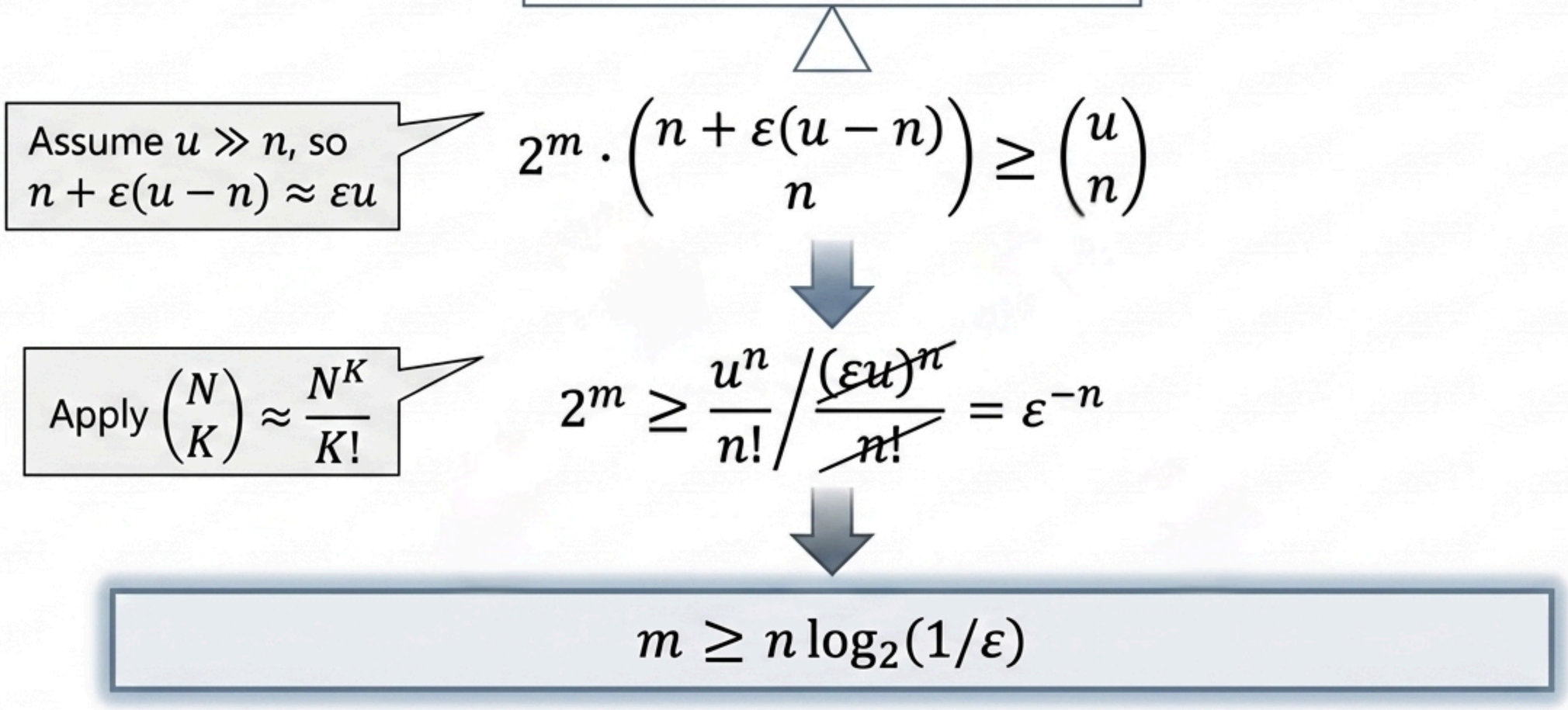
False positives mean one state covers multiple sets.

$$|A| \leq n + \epsilon(u - n)$$

A single m-bit state can safely represent at most $\binom{n + \epsilon(u-n)}{n}$ different sets.

Deriving the Information-Theoretic Limit

Total States × Sets per State $\geq$ Total Possible Sets



The unbreakable floor. No approximate membership data structure can exist below this line.

The Bloom Filter Gap: Calculating the Space Overhead

The Optimal Reality: Bloom filter minimizes error at $\varepsilon \approx (0.6185)^{m/n}$

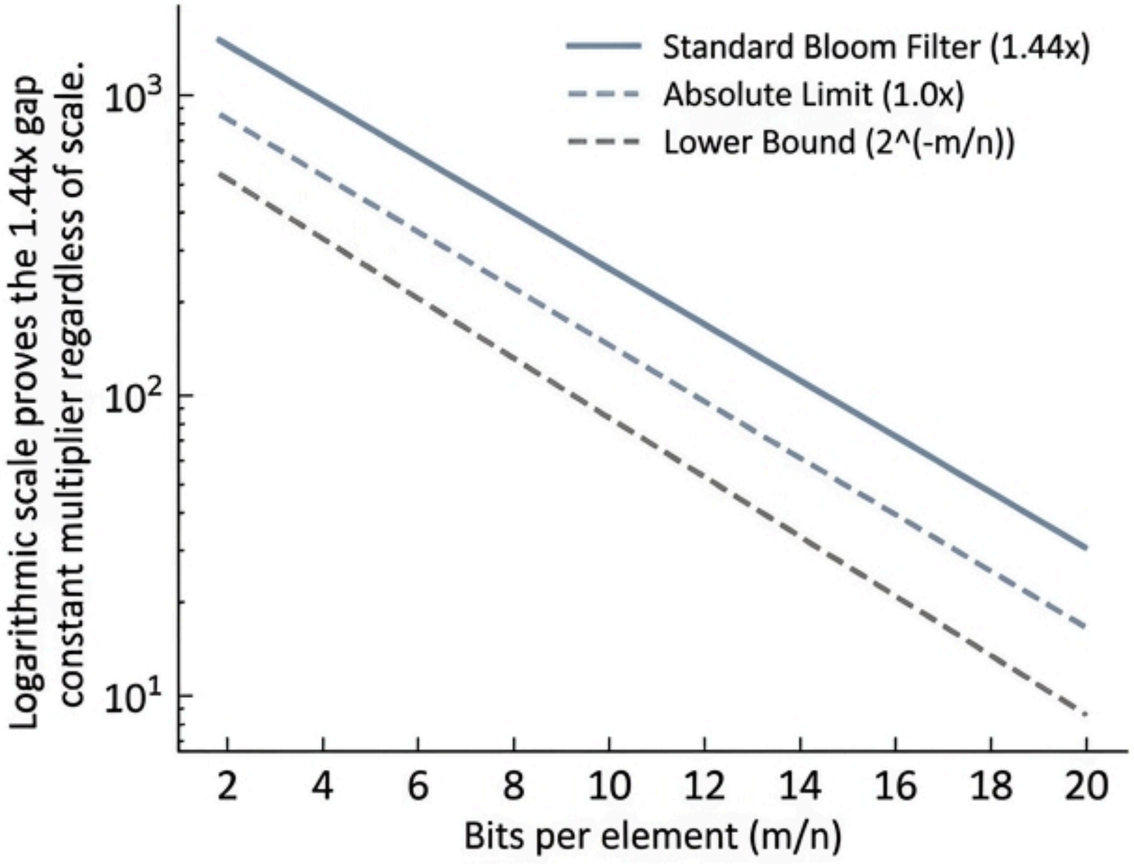
↓

Solving for m yields: $\frac{m}{n} = \frac{\ln(1/\varepsilon)}{(\ln 2)^2}$

↓

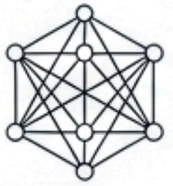
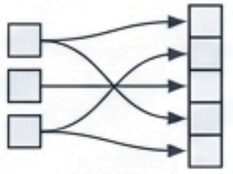
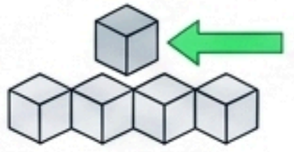
$$m = \frac{n \log_2(1/\varepsilon)}{\ln 2} \approx 1.44n \log_2(1/\varepsilon)$$

An optimal Bloom filter requires 1.44 times the space of the absolute information-theoretic minimum. This represents a 44% space overhead.



Why a 44% Space Penalty is an Engineering Triumph

Why do system architects willingly pay a 1.44x space penalty? Because achieving 1.0x breaks dynamic usability.

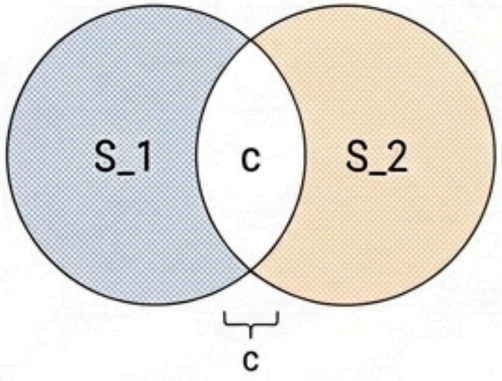
	Information-Theoretic Perfect Hashing	Standard Bloom Filters
Space Overhead	**1.0x** (Theoretical limit achieved) $m = n \log_2(1/\varepsilon)$	**1.44x** (The 44% penalty) $m \approx 1.44\, n \log_2(1/\varepsilon)$
Complexity & Pre-computation	 **Static Set S Knowledge Needed** **Massive.** Requires knowing the entire set S in advance to find collision-free functions.	 **Simple, Independent Hashes** **Zero.** Uses simple, independent hash functions with no required knowledge of data distribution.
Dynamic Nature	 **Re-computation for each update** **Static.** Inserting a single new element requires an expensive, complete system re-computation.	 **O(1) Insertions & Updates** **Highly Dynamic.** Seamless O(1) insertions without disrupting the structure.

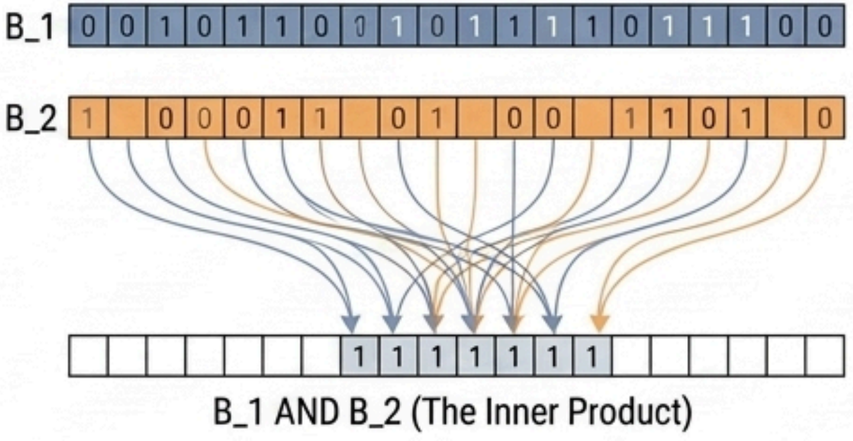
The 1.44 factor is a negligible price. We trade a fractional amount of memory to gain O(1) speed, zero pre-computation, and seamless dynamic updates. Bloom filters are within a constant factor of mathematical perfection while remaining infinitely deployable.

Isolating the Intersection: The Bitwise AND Challenge

To find the overlap of two sets ($c = |S_1 \cap S_2|$) without computing the union, we must analyze the probability of bits colliding in their inner product.

The Visual Architecture





B_1 AND B_2 (The Inner Product)

[SYS_CHECK] Assumption: Both filters utilize identical array length m and identical k hash functions.

The Algebraic Blockade

A bit is set to 1 in the inner product in one of two mutually exclusive ways:

1. Directly via the intersection: The bit is set by an element belonging to $S_1 \cap S_2$.

2. Coincidentally via distinct elements: The bit is set independently by a unique element in S_1 and a unique element in S_2.

The massive heroic equation, using mathematical equation:

$$\mathrm{E}[1\text{s in AND}] = m\left(1 - \left(1 - \frac{1}{m}\right)^{k|S_1|} - \left(1 - \frac{1}{m}\right)^{k|S_2|} + \left(1 - \frac{1}{m}\right)^{k(|S_1|+|S_2|-c)}\right)$$

The Algebraic Blockade: Solving this complex equation directly for the unknown variable c is exceedingly difficult. We require a mathematical pivot.

The Zero-Counting Trick: Simplifying the Math

Just as analyzing empty bins is easier in probability, observing zero bits allows the unknown set sizes to beautifully cancel out.

Step 1: Inclusion-Exclusion of Zeros

$$Z_{12} = Z_1 + Z_2 - Z_v$$

- Z_{12}: Zeros observed in the AND filter (Inner Product)
- Z_1, Z_2: Zeros observed in B1 and B2
- Z_v: Zeros expected in the OR filter (Union)

Step 2. By substituting the theoretical expected values (justified by Azuma-Hoeffding concentration bounds), Broder and Mitzenmacher isolate the intersection using a simple ratio.

Theoretical Expected Values:

$$Z_1 \approx m(1 - \tfrac{1}{m})^{k|S_1|}$$
$$Z_2 \approx m(1 - \tfrac{1}{m})^{k|S_2|}$$
$$Z_v \approx m(1 - \tfrac{1}{m})^{k|S_1 \cup S_2|}$$

$$\frac{Z_v}{Z_1 Z_2} = \frac{m(1 - \frac{1}{m})^{k|S_1 \cup S_2|}}{m(1 - \frac{1}{m})^{k|S_1|} \cdot m(1 - \frac{1}{m})^{k|S_2|}}$$

The Magic Cancellation: Because $|S_1 \cup S_2| - |S_1| - |S_2| = -c$, the known set sizes vanish entirely.

$$\frac{Z_1 + Z_2 - Z_{12}}{Z_1 Z_2} \approx \frac{1}{m}(1 - \frac{1}{m})^{-kc}$$

Step 3. The Operational Formula

Solving for c:

$$c \approx (-\frac{1}{k}) \cdot \frac{\left(m \cdot \frac{Z_1 + Z_2 - Z_{12}}{Z_1 Z_2}\right)}{\ln(1 - \frac{1}{m})}$$

Recovering the Intersection: A Concrete Simulation

Proving the operational validity of the zero-counting formula in a simulated 10,000-bit environment.

[01] SYSTEM CONFIGURATION

Array length:	m = 10,000 bits
Hash functions:	k = 4
Set sizes:	|S1| = 1,000
	|S2| = 1,000

True Intersection:

c = 500 shared elements

(Implied True Union Size = 1,500)

[02] STATISTICAL EXPECTATIONS

If we count the zero bits in our arrays, probability dictates they will tightly match these values:

$$Z1 \approx 10{,}000\left(1 - \frac{1}{10{,}000}\right)^{4000} \approx 6{,}703 \text{ zeros}$$

$$Z2 \approx 6{,}703 \text{ zeros}$$

$$Z_V \text{ (Union)} \approx 10{,}000\left(1 - \frac{1}{10{,}000}\right)^{6000} \approx 5{,}488 \text{ zeros}$$

Deriving the Inner Product:

Therefore, Z12 must be:

6,703 + 6,703 − 5,488 = 7,918

[03] MATHEMATICAL RECOVERY

Pretending we only know the observed zeros (Z1, Z2, Z12), we apply our operational formula.

Step A (The Ratio):

$$\frac{6703 + 6703 - 7918}{6703 \times 44{,}930{,}209} = \frac{5488}{0.000122145}$$

Step B (Solving for c):

$$c \approx \left(\frac{-1}{4}\right) \cdot \frac{\ln(10{,}000 \cdot 0.000122145)}{\ln(0.9999)}$$

$$c \approx -0.25 \cdot \frac{0.199997}{-0.000100005}$$

$$c \approx 499.96$$

The formula accurately recovers the **exact intersection** size of 500 **elements** using nothing but bitwise zero-counting.

Bloom Filter Variants

Counting Bloom Filters

Bits $\longrightarrow$ Counters

Safely supports element deletions in dynamic sets without risking hash collision corruption.

Compressed Bloom Filters

Dense Array $\rightarrow$ Sparse Array + Compressor

Intentionally lowers entropy to optimize transmission bandwidth (z) over uncompressed size (m).

Spectral Bloom Filters

Boolean Output $\rightarrow$ Frequency Estimation

Maintains independent item frequencies while mathematically bounding overestimation errors.

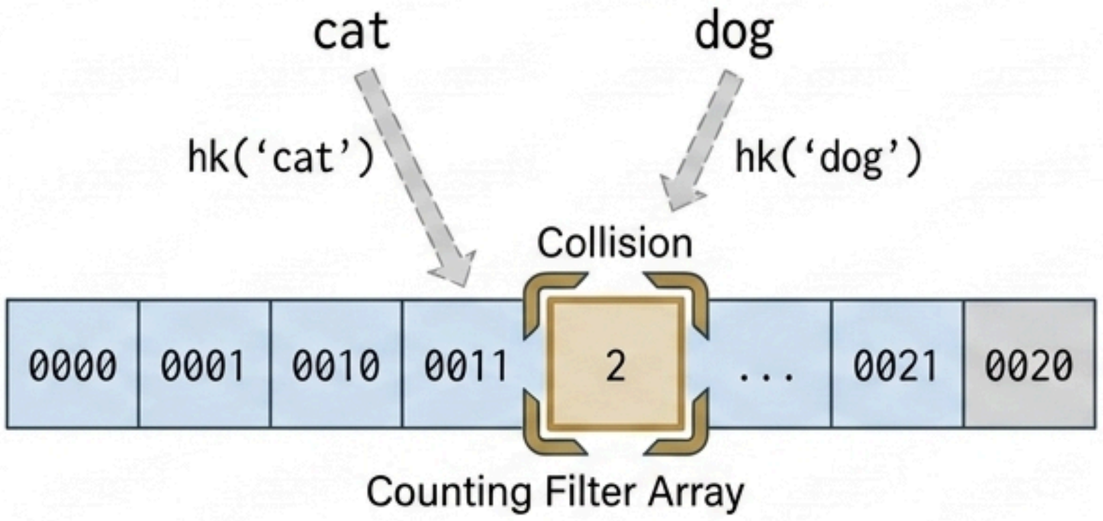
Scalable Bloom Filters

Fixed Capacity $\rightarrow$ Dynamic Growth

Sequentially instantiates overlapping matrices bounded by a maximum false positive rate as capacity is reached.

Mechanics: Counting & Compressed Variants

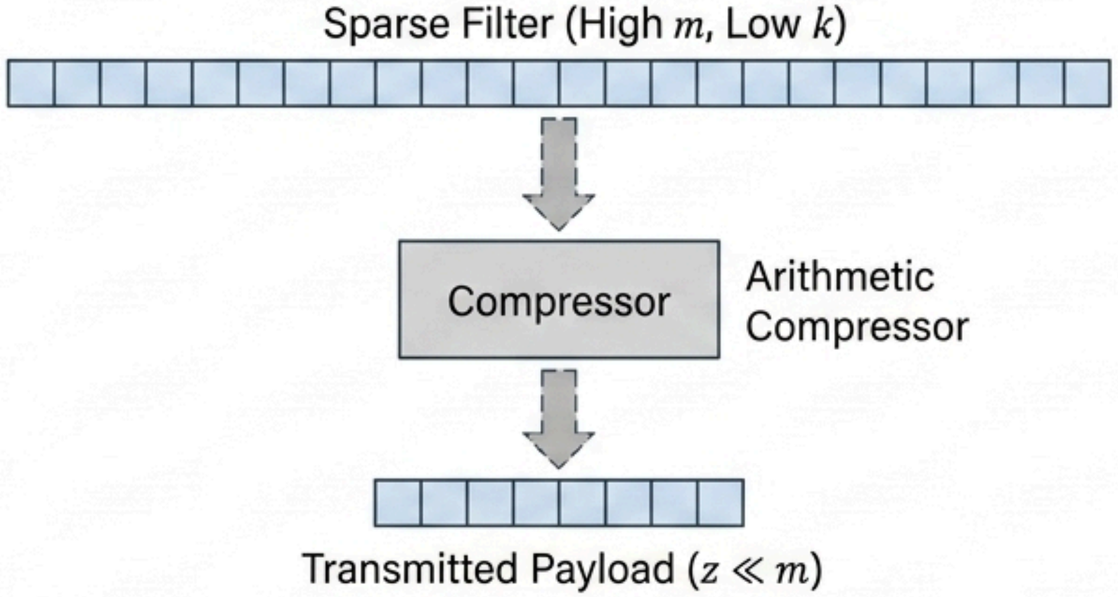
Bits to Counters



Instead of bits, buckets hold integers (typically 4 bits each). To query an element, we hash it and check its corresponding buckets. If buckets have different values due to collisions, we choose the minimum value as the element's frequency count.

When a collision occurs during insertion, the counter increments safely from 1 to 2, allowing for future deletions by decrementing.

Optimizing Bandwidth

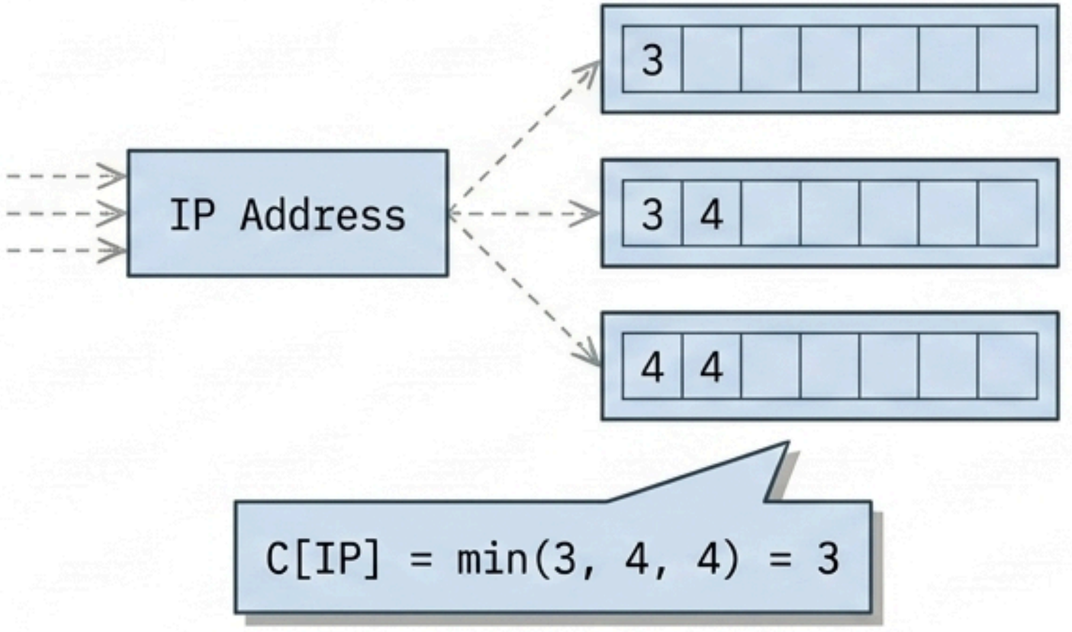


A larger but sparser filter is highly compressible. The compressor significantly reduces the size of the filter into a smaller payload for network transmission.

Upon receipt, the payload is decompressed to reconstruct the original, larger filter for normal operations, maintaining the low false positive rate (f) within the bandwidth budget.

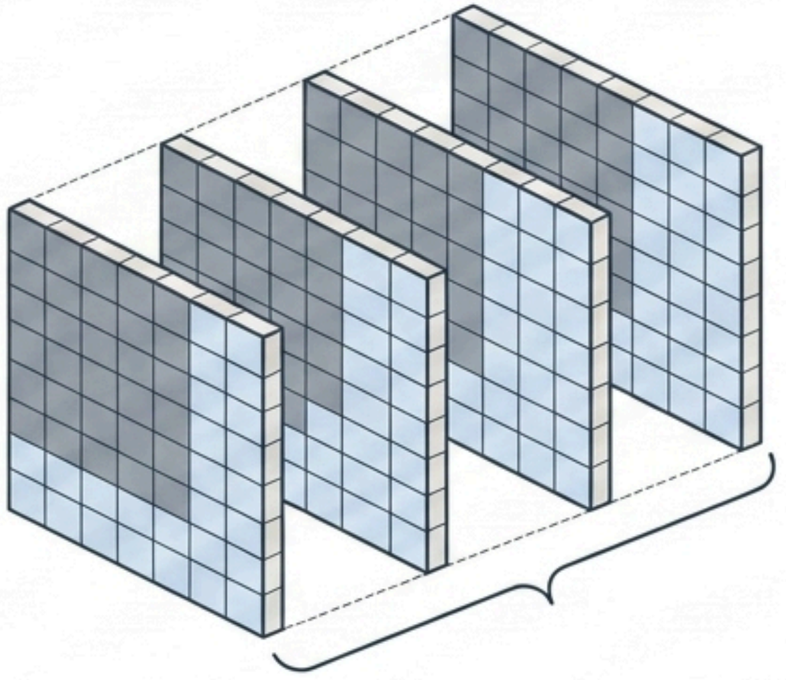
Mechanics: Spectral & Scalable Variants

Frequency Estimation



Because hash collisions can only add to counters and never subtract, isolating the minimum bucket value provides a mathematically guaranteed upper bound on item frequency.

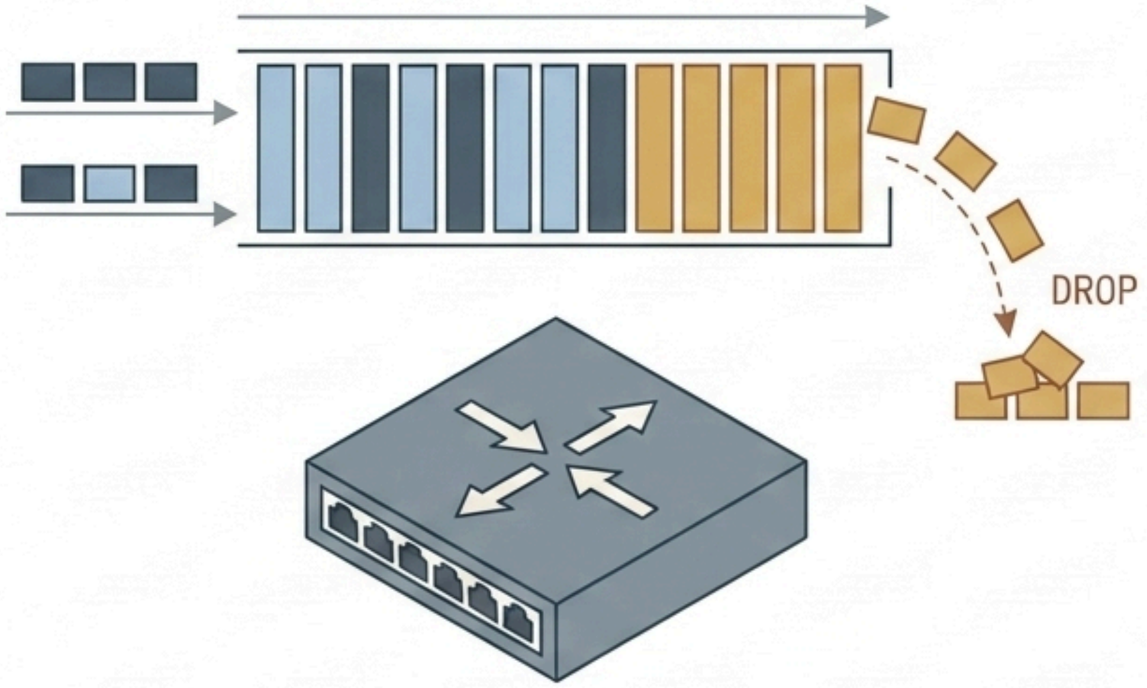
Dynamic Capacity Expansion



When capacity ‘n’ is reached, standard filters fail. Scalable variants create a new, parallel filter with a progressively tighter false-positive boundary. These new filters are geometrically larger (e.g., 2x, 4x, 8x capacity) to ensure the cumulative ‘f’ rate remains stable over time.

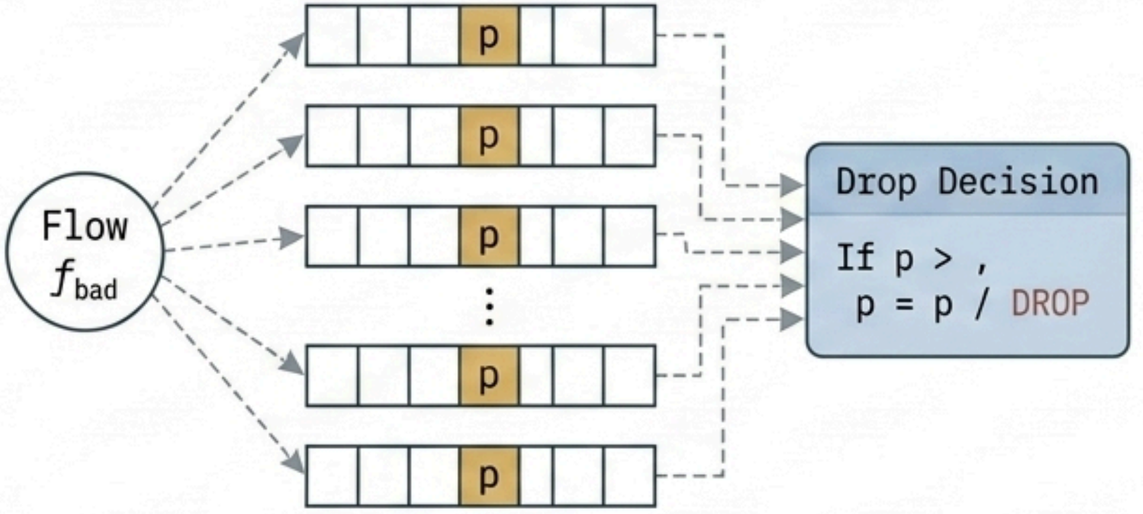
Queue Management & Stochastic Fair Blue (SFB)

The Problem: Congestion & Queue Management



Traditional “tail-drop” queues treat all flows equally. Aggressive flows fill the buffer, causing congestion and punishing well-behaved traffic. This leads to unfairness and inefficiency.

The Solution: Stochastic Fair Blue (SFB)

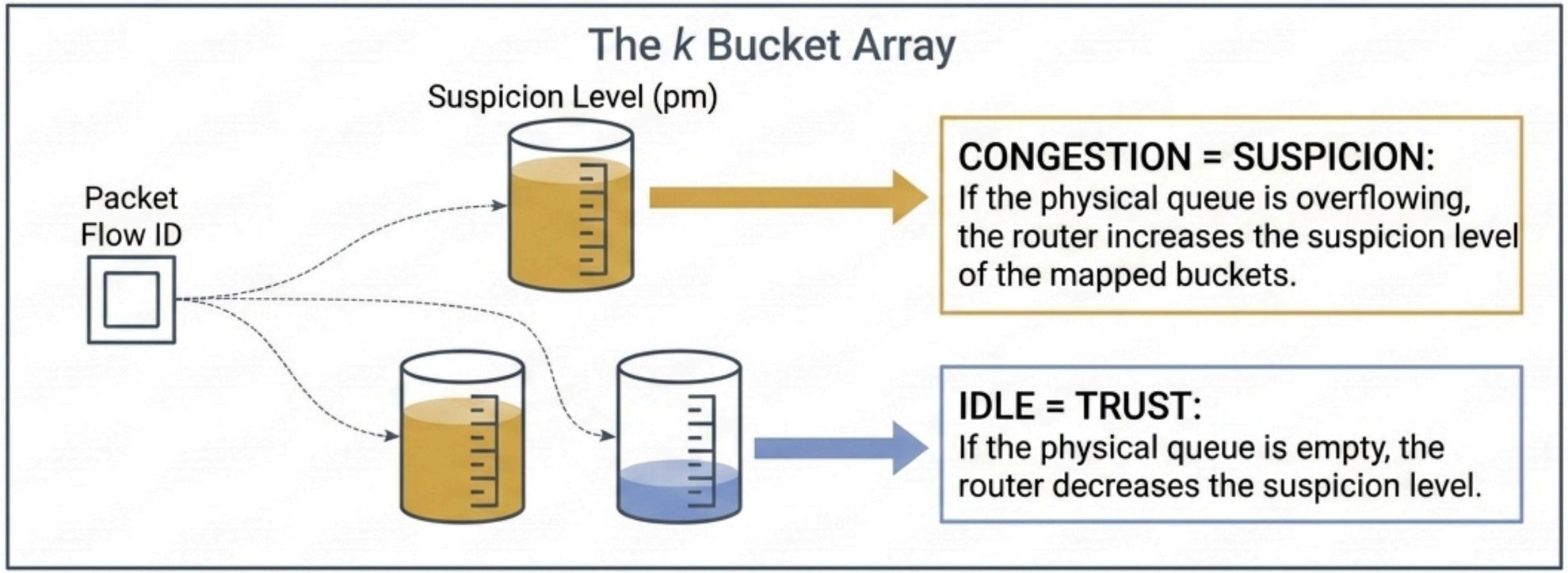


SFB uses a multi-level hashing scheme (like a Bloom filter) to probabilistically identify and drop packets from non-responsive flows based on bin marking probabilities.

Real-World Application: Used in routers for Active Queue Management (AQM) to ensure fairness, prevent congestion collapse, and protect sensitive traffic.

The Solution — Introducing Stochastic Fair Blue (SFB)

SFB utilizes a Counting Bloom Filter to identify bad flows probabilistically, eliminating the need for absolute Flow IDs.

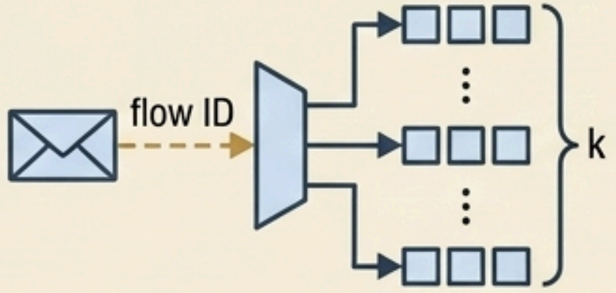


The Core Insight: A non-responsive, aggressive flow will continuously flood every bucket it touches. Over time, it drives the suspicion level of all its mapped buckets to nearly 100%.

Algorithm Mechanism — The 3-Step Filtering Pipeline

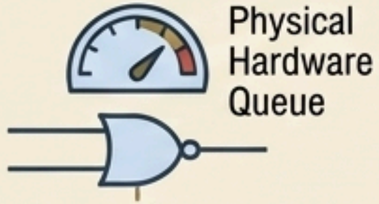
The State: The Router maintains **k** levels of accounting bins. Each bin tracks **pm**: the marking probability (likelihood that traffic in this bin should be dropped).

Step 1: Packet Arrival (Extraction & Hashing)



A packet arrives. The router extracts its Flow ID (e.g., Source/Dest IP) and processes it through k independent hash functions. The packet is mathematically mapped to exactly k specific bins.

Step 2: Update (The Learning Phase)



A

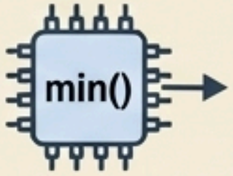
Queue Full -> Increment pm in the hashed bins.

B

Queue Empty -> Decrement pm in the hashed bins.



Step 3: Decision (The Filtering Phase)



To decide the packet's fate, the router evaluates the pm values of all k mapped bins. It calculates the final drop probability using the minimum value:

$$P_{drop} = \min(pm1, pm2, ..., pmk)$$

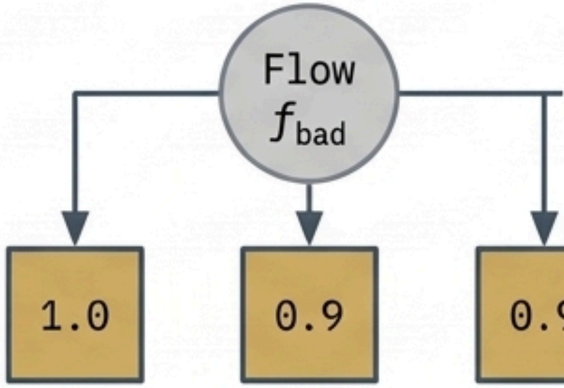
Drop

Forward

Comparison Matrix — The Offender vs. The Victim

The Safety Valve: Why $P_{drop} = \min(\ldots)$? The minimum function acts as an absolute safeguard, ensuring hash collisions do not destroy well-behaved traffic.

The Offender (f_{bad})



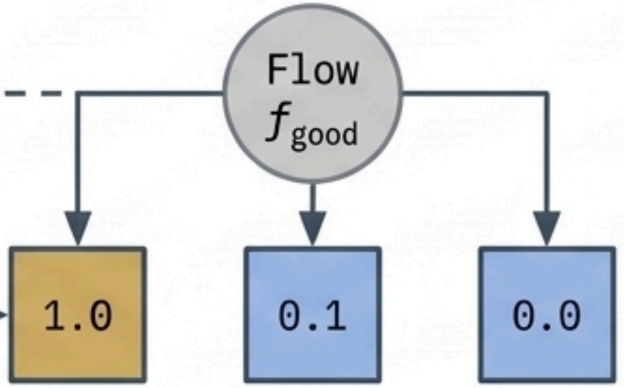
A non-responsive flow sends a massive volume of traffic, polluting all k of its bins.

DECISION (f_{bad})

$\min(1.0, 0.9, 0.9) = 0.9 \rightarrow$ DROP ❌

Result: The drop probability is high. The flow is rightfully penalized.

The Victim (f_{good})



A well-behaved flow happens to share one hash bin with a bad flow (a collision). Because the flow itself isn't aggressive, its other k-1 bins remain healthy.

DECISION (f_{good})

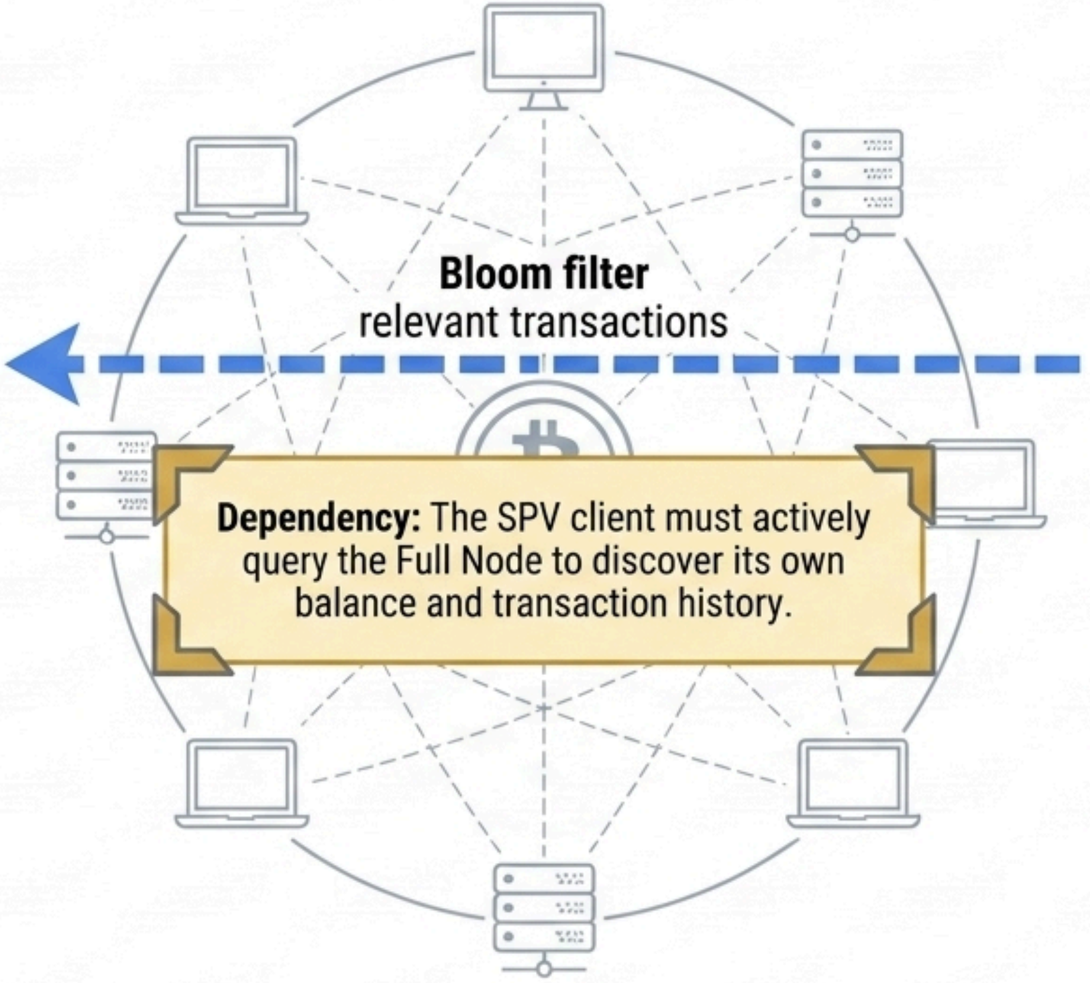
$\min(1.0, 0.1, 0.0) = 0.0 \rightarrow$ FORWARD ✅

Result: The polluted bin is effectively ignored. Good traffic is forwarded unharmed.

The Blockchain Ecosystem: Full Nodes vs. Light Clients

FULL NODES

- **Role & Storage:** Maintains the complete, append-only ledger (often hundreds of gigabytes).
- **Verification:** Independently verifies every block, transaction signature, and consensus rule.
- **Trust Model:** Ensures the network's security without trusting third parties. Highly secure, but resource-intensive.



LIGHT NODES (SPV)

- **Role & Storage:** Designed for resource-constrained devices (mobile, IoT). Downloads only block headers—a tiny fraction of total data.
- **Verification:** Uses Simplified Payment Verification (SPV) to verify transaction inclusion without processing the whole chain.
- **Trust Model:** Relies entirely on connected Full Nodes to retrieve specific transaction data relevant to their wallets.

The SPV Query Problem: The Privacy-Bandwidth Dilemma

The Core Question:
A Light Node must ask a remote Full Node: “Do you have any transactions for my address X?”

Trade-off Spectrum

NAIVE APPROACH 1
Bandwidth Optimization

- **Mechanism:** The Light Client explicitly sends its exact list of addresses to the Full Node.
- **Bandwidth Status:** Optimal. The client downloads only the exact data it needs.
- **Privacy Status (Catastrophic):** The Full Node learns exactly which addresses the user owns. Complete privacy loss.

?

The Gap: SPV requires a mechanism that balances these extremes without breaking the lightweight architecture.

NAIVE APPROACH 2
Privacy Optimization

- **Mechanism:** The Light Client downloads every transaction in the network and filters them locally.
- **Privacy Status:** Optimal. The Full Node learns nothing about the user’s specific addresses.
- **Bandwidth Status (Catastrophic):** Massive bandwidth consumption. This entirely defeats the purpose of a Light Node.

The Solution: BIP 37 Bloom Filters & Plausible Deniability

The 4-Step BIP 37 Filtering Mechanism

1. Filter Construction: The Light Client creates a Bloom filter containing its addresses, public keys, and TX IDs. Crucially, the client mathematically tunes the false positive rate ('p') to balance its needs.

2. Transmission: The client transmits this custom Bloom filter to a remote Full Node over the network.

3. Server-Side Filtering: The Full Node scans new transactions and historical blocks against the provided filter.

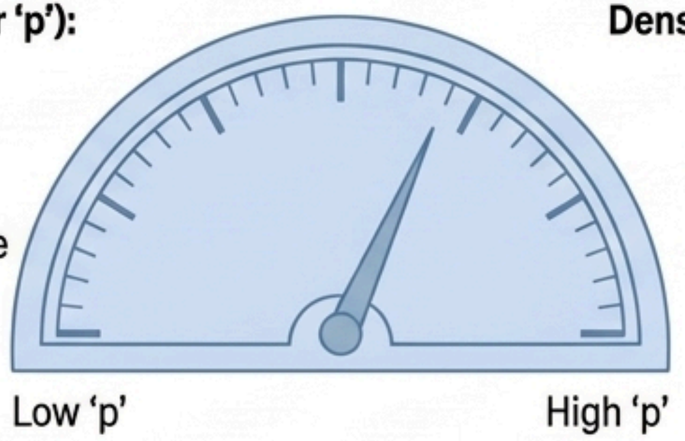
4. Data Retrieval: If a transaction matches the filter (true match or false positive), the Full Node transmits it back to the client.

False Positives as a Feature (Plausible Deniability)

- **The Insight:** Bloom filter errors are deliberately engineered as a privacy shield.
- Because the filter allows false positives, the Full Node returns real transactions mixed with irrelevant ones.
- The Full Node cannot mathematically distinguish which returned transactions truly belong to the client and which are random collisions.

Tuning Dial (p)

Sparser Filter (Lower 'p'): Saves bandwidth by minimizing irrelevant data, but leaks more specific information about the user's interests.



Denser Filter (Higher 'p'): Provides more camouflage (higher privacy) but incurs a higher bandwidth cost (downloading irrelevant data).

Probabilistic data structures trade an acceptable, one-sided error for otherwise impossible space effciencies.

wherever exactness is expensive and space is constrained, the Bloom filter remains a near-optimal solution.

Questions?